

NENA

Eight-Person Delivery Plan

A balanced, collision-resistant plan for delivering the Android MVP quickly and efficiently.

8	10	7 DAYS
parallel owners	points each	MVP target

1. Current project status

NENA is currently an advanced proof of concept, not yet a production-ready application.

What already exists

- A Flutter UI design with home, avatar, settings, and calling screens.
- Camera capture and client service classes for sign prediction.
- Local Swahili text-to-speech with male/female voice profile selection.
- A CLIP-based few-shot sign-recognition notebook and reference sign media.
- A Whisper-based Swahili speech-transcription prototype.
- A WebRTC/VoIP service design and call user interface.
- 132 sign video files (68 .mp4 and 64 .MOV), plus keyframe images and avatar assets.
- Concept, architecture, guideline, and presentation documents.

What remains incomplete

- The Flutter files are not yet inside a complete Flutter project; configuration such as pubspec.yaml, platform folders, dependency locking, and build instructions is absent.
- Flutter backend URLs are placeholders.
- WebRTC signalling is a stub and has no working signalling/TURN deployment.
- The avatar is an animated placeholder rather than validated Tanzanian Sign Language output.
- The AI code is notebook-oriented and has not been packaged as a reproducible, tested backend service.
- Dataset labels, splits, quality checks, accuracy benchmarks, and model acceptance thresholds are not finalized.
- Automated unit, widget, integration, performance, accessibility, and field tests are absent.
- An ngrok authentication token is embedded in nena_project_updated.ipynb; revoke it immediately, remove it from the notebook, and use environment secrets.

2. Delivery target

Build a testable Android MVP that demonstrates both directions:

- 1 Camera sign -> Swahili token -> selected Swahili voice.
- 2 Spoken Swahili -> transcript/keywords -> ordered sign video or avatar playback.
- 3 One working in-app WebRTC call between two test devices using those pipelines.

The MVP is complete only when it builds from a clean checkout, passes automated tests, and completes a scripted end-to-end demonstration on two Android devices.

3. Equal allocation

Each person receives 10 delivery points. Points measure expected work, not the number of files. Every assignment includes implementation (6 points), tests and verification (2 points), and documentation/review (2 points).

Person	Workstream	Exclusive ownership	Deliverables (10 points)	Depends on
1	Flutter foundation and release build	mobile/pubspec.yaml, mobile/android/ mobile/lib/main.dart, build scripts	Create the proper Flutter project; move existing Dart code into it; configure dependencies, permissions, assets, environment configuration, linting, and debug/release builds; document clean setup.	None
2	Sign dataset and evaluation	data/, tools/data/, dataset manifests	Inventory and rename media safely; create label manifest, signer metadata policy, train/validation/test split, duplicate/corrupt-file checks, frame extraction, and a reproducible evaluation report. Do not alter source media; generate normalized copies.	None
3	Sign-recognition model	backend/sign/ models/sign/	Convert the CLIP experiment into modules; train/build prototypes from Person 2's manifest; define confidence/rejection behavior; expose predict(frame); benchmark accuracy and latency; export versioned model artifacts.	Person 2's manifest contract
4	Speech, keyword, and TTS pipeline	backend/speech/, mobile/ lib/services/whisper_service.dart, mobile/lib/services/tts_service.dart	Package Whisper transcription and Swahili keyword ordering; implement audio validation and errors; finalize male/female TTS profiles; test with test_habari.wav and additional samples; document supported vocabulary.	API contract only
5	Backend API, deployment, and security	backend/api/ backend/deploy/ .env.example, API tests	Build one persistent FastAPI/Flask service for health, sign prediction, and transcription; add schemas, validation, logs, CORS/auth policy, Docker/runtime setup, secret handling, and deployment guide. Revoke/remove the exposed ngrok token.	Persons 3 and 4 provide callable modules
6	Mobile UX and accessibility	mobile/lib/screens/ mobile/lib/widgets/ mobile/lib/providers/ assets manifest	Integrate camera flows, captions, avatar/video dictionary, settings persistence, loading/offline/error states, accessibility labels, large-text support, and Deaf-user-friendly feedback. Add widget tests. Must not edit service implementations.	Person 1 scaffold and agreed service interfaces
7	Real-time calling	mobile/lib/services/voip_service.dart, realtime/	Implement persistent WebSocket signalling, WebRTC offer/answer/ICE handling, STUN/TURN configuration, reconnect/hang-up behavior, text-token data channel, and a two-device call demo. Add signalling and call-state tests.	Person 1 scaffold; API/token contracts
8	QA, integration, documentation, and field validation	test/, integration_test/ docs/, CI workflow	Create the test matrix and CI; write API contract tests and end-to-end mobile tests; measure latency, failure recovery, accessibility, and device compatibility; coordinate Deaf-user validation; maintain defect register and release checklist. Does not fix owners' code directly—files defects for owners.	All streams provide testable increments

4. Interfaces to freeze on Day 1

The team must agree on these contracts before parallel implementation. Changes require approval from affected owners.

Sign prediction

POST /v1/sign/predict

Request: JPEG image as multipart upload.

Response:

```
{
  "token": "habari",
  "display_text": "Habari",
  "confidence": 0.91,
  "model_version": "sign-0.1.0",
  "accepted": true
}
```

Speech transcription

POST /v1/speech/transcribe

Request: WAV/M4A audio as multipart upload.

Response:

```
{
  "text": "Habari yako",
  "tokens": ["habari", "yako"],
  "language": "sw",
  "model_version": "speech-0.1.0"
}
```

WebRTC token message

```
{
  "type": "sign_token",
  "token": "habari",
  "sequence": 12,
  "timestamp_ms": 1786700000000
}
```

Shared error response

```
{
  "error": {
    "code": "LOW_CONFIDENCE",
    "message": "Sign could not be recognized confidently",
    "retryable": true
  }
}
```

5. Collision-prevention rules

- 1 Create one integration branch and eight feature branches: codex/foundation, codex/data, codex/sign-model, codex/speech, codex/backend, codex/mobile-ux, codex/realtime, and codex/qa.
- 2 Only the listed owner edits files in their ownership area. Cross-area changes are requested through a small issue or pull request comment.
- 3 Never commit directly to the integration branch. Each deliverable uses a small pull request reviewed by Person 8 and one technically relevant owner.
- 4 Do not store generated models, normalized videos, secrets, or local environments in Git. Use artifact storage and document checksums/versions.
- 5 Freeze API schemas in docs/contracts/ on Day 1. Generate fixtures from them so mobile and backend development can proceed independently.
- 6 Person 6 uses mocked service interfaces until Persons 4, 5, and 7 are ready. Person 8 tests contracts rather than internal implementations.
- 7 Rebase or merge the integration branch into feature branches daily. Resolve ownership conflicts with the file owner, never by overwriting their work.
- 8 Every pull request must include tests, setup notes, evidence of the result, and any known limitation.

6. Fast execution schedule

Day 0: security and repository preparation

- Person 5 revokes the exposed ngrok token and prepares secret handling.
- Person 1 creates the Flutter scaffold and repository layout.
- Person 8 creates the issue board, acceptance checklist, and CI skeleton.

Day 1: contracts and baselines

- Freeze API and WebRTC message contracts.

- Person 2 publishes the first dataset manifest and split.
- Persons 3 and 4 publish baseline accuracy/latency results.
- Person 6 connects the UI to mock services.
- Person 7 proves a basic two-device WebRTC connection.

Days 2-4: parallel implementation

- All owners deliver one tested pull request per day where practical.
- Integrate using fixtures and mocks; do not wait for every real component.
- Person 8 runs CI and logs blockers twice daily.

Day 5: first integrated MVP

- Connect the Flutter app to the deployed backend.
- Run sign-to-speech and speech-to-sign tests.
- Test the first translated two-device call.

Days 6-7: stabilization and demonstration

- Fix critical defects, test weak-network behavior, measure end-to-end latency, and run accessibility checks.
- Complete the demonstration script, installation guide, model card, privacy notes, and release checklist.

If model quality requires more data, keep the seven-day target as a bounded-vocabulary MVP and schedule vocabulary expansion as the next sprint.

7. Shared definition of done

A person's 10 points are complete only when:

- Their deliverable works from a clean checkout with documented commands.
- Relevant automated tests pass.
- No secret or personal participant data is committed.
- Interfaces match the frozen contracts.
- Performance or accuracy evidence is recorded where applicable.
- Person 8 and one peer have reviewed the pull request.
- Known limitations and next actions are documented.

8. Daily coordination

Use a 15-minute daily meeting with only four questions per person:

- 1 What was merged?
- 2 What will be merged today?
- 3 Is any frozen interface at risk?
- 4 What blocker needs an owner now?

Track tasks in columns: Ready, In progress, Review, Integration test, and Done. Each person may have only one primary item in progress, which reduces unfinished work and makes blockers visible early.